面试指南之接口参数

以项目 `rosa-bookshelf` 为例，梳理前端调用接口时常见的参数类型与在本项目中的具体用法，包括 Header、Body、Query 以及响应处理策略。内容兼顾通识与项目落地实践，便于面试与日常开发参考。

一、概念总览

- Header 参数：随 HTTP 请求发送的元数据，不参与业务实体本身，常用于认证、版本、语言等。
- Body 参数：请求体内容，常用于提交或更新资源（POST/PUT/PATCH），可为 `application/json` 或 `multipart/form-data` 等。
- Query 参数：URL 查询串中的键值，如分页、筛选、删除定位等（常见于 GET/DELETE）。
- Response：服务端返回结果与状态码；本项目通过业务码 `code` 统一判断成功与否。

二、Header 参数

作用与特点

- 携带认证信息（如 `Authorization`）、版本、语言、幂等键等。
- 用于服务端网关、鉴权、路由或限流；不会出现在 URL 或请求体中。

项目中的做法

- 在请求拦截器中按需设置 `Authorization`（来源于用户状态）：
  - 位置：`src/utils/request.js:18-31`，当 `useUserStore().token` 存在时设置 `config.headers.Authorization = userStore.token`（`src/utils/request.js:24-27`）。

如何新增或定制 Header（示例）

1. 单次请求添加（仅该接口需要时，推荐）：

```
await request.get('/my/article/list', {
  params: { pagenum: 1, pagesize: 10 },
  headers: { 'X-Trace-Id': traceId }
})
```

2. 设置默认 Header（多数接口都需要时）：

```
// 初始化处或拦截器中：
request.defaults.headers.common['X-Client-Version'] = 'web-1.2.3'
// 或在拦截器：
// config.headers['X-Client-Version'] = 'web-1.2.3'
```

3. API 封装层开放 `config` 透传（偶尔定制时）：

```
export const foo = (data, config = {}) => request.post('/path', data,
config)
// 调用：foo(payload, { headers: { 'X-Trace-Id': traceId } })
```

提示：若后端要求 `Bearer <token>` 格式，确保 `userStore.token` 自带前缀或在拦截器统一拼接。

三、Body 参数

概念与用途

- 请求体内容，用于提交业务数据或文件，常见于 POST/PUT/PATCH。

类型与项目示例

1. `application/json`

- 直接传对象，Axios 自动序列化为 JSON。
- 示例：注册与登录（`src/api/user.js:5-8, 10-12`）、更新资料（`src/api/user.js:18-23`）、更新头像（`src/api/user.js:24-25, 28-32`）。

```
// 登录
await userLoginService({ username, password })

// 更新资料
await userUpdateUserInfoService({ id, username, nickname, email, user_pic })
```

2. `multipart/form-data`

- 用于文件上传或包含二进制；通过 `FormData` 封装。
- 示例：文章发布与更新（`src/api/article.js:29-31, 36-38`）。

```
const fd = new FormData()
fd.append('title', title)
fd.append('content', content)
fd.append('cover_img', file) // File 或 Blob
```

```
await artPublishArticleService(fd)
```

注意：发送 `FormData` 时不要手动设置 `Content-Type: multipart/form-data`，浏览器会自动带 boundary；手动设置可能导致上传失败。

四、Query 参数

概念与用途

- 拼接在 URL 的查询串上（`?key=value`），用于筛选、分页、删除定位等；Axios 通过 `params` 传递，自动序列化。

项目示例

```
// 获取文章列表（分页筛选）
await artGetArticleListService({ pagenum: 1, pagesize: 10, cate_id: '',
state: '' })
// 对应：src/api/article.js:17-27（通过 { params: { ... } } 传参）

// 删除分类/文章（通过 id 定位）
await artDelChannelsService(id)       // src/api/article.js:13-16
await artDeleteArticleService(id)     // src/api/article.js:42
```

提示：GET 不要放请求体；DELETE 若后端强制需要 body，可用 `{ data }`，但更常见仍用 `params`。

五、响应处理

项目中的约定与拦截器逻辑

- 成功：`res.data.code === 0` 视为业务成功（`src/utils/request.js:35-38`）。
- 业务失败：弹错误消息并拒绝 Promise（`src/utils/request.js:39-45`）。
- 401 未授权：清除 token、跳转登录并提示（`src/utils/request.js:53-58`）。

使用建议

```
// 成功取数
const res = await userGetUserInfoService()
const data = res.data.data

// 失败捕获
try {
  await artDeleteArticleService(id)
```

```
} catch (err) {
  // 可查看 err.response?.status 与 err.message
}
```

响应类型与其他情况

- 本项目 API 响应统一为 `application/json`（包括 `{}` 空对象）。
- 可能的其他类型（项目当前未用）：文件下载 `blob` / `arraybuffer`、纯文本 `text/plain`。

```
// 文件下载示例（如有需要）
await request.get('/export', { params: { ... }, responseType: 'blob' })
```

- 若后端返回 `204 No Content`，无响应体（无 `code` 字段）；需在拦截器或调用处额外处理。

六、项目用法速览（摘录）

```
// 文章分类
export const artGetChannelsService = () => request.get('my/cate/list')
// src/api/article.js:3-4
export const artAddChannelsService = (data) => request.post('/my/cate/add',
data)            // src/api/article.js:9-11
export const artUpdateChannelsService = (data) =>
request.put('/my/cate/info', data)         // src/api/article.js:6-8
export const artDelChannelsService = (id) => request.delete('/my/cate/del',
{ params: { id } }) // src/api/article.js:13-16

// 文章管理
export const artGetArticleListService = ({ pagenum, pagesize, cate_id, state
}) =>
  request.get('/my/article/list', { params: { pagenum, pagesize, cate_id,
state } })         // src/api/article.js:17-27

export const artPublishArticleService = (data) =>
request.post('/my/article/add', data)      // multipart/form-data,
src/api/article.js:29-31
export const artGetArticleDetailService = (id) =>
request.get('/my/article/info', { params: { id } }) //
src/api/article.js:33-35
export const artUpdateArticleService = (data) =>
request.put('/my/article/info', data)       // multipart/form-data,
src/api/article.js:36-38
export const artDeleteArticleService = (id) =>
request.delete('/my/article/info', { params: { id } }) //
```

```
src/api/article.js:42

// 用户
export const userRegisterService = async ({ username, password, repassword
}) =>
  request.post('api/reg', { username, password, repassword })
// src/api/user.js:5-8

export const userLoginService = async ({ username, password }) =>
  request.post('api/login', { username, password })
// src/api/user.js:10-12

export const userGetUserInfoService = async () => request.get('my/userinfo')
// src/api/user.js:14

export const userUpadteAvatarService = (avatar) =>
request.patch('/my/update/avatar', { avatar }) // src/api/user.js:24-25, 28-
32
```

七、进阶与注意事项

- 统一“摘取核心数据”：当前拦截器返回 `res` 对象；如希望统一返回 `res.data`，可在拦截器改为 `return res.data`，并在业务层按新约定取值。
- 401 提示的重复弹窗：当前逻辑在 401 分支后仍会弹“Other Error”一次；如遇双提示，可在 401 分支后直接 `return Promise.reject(err)` 终止后续提示。
- 令牌格式：确认后端期望的 `Authorization` 规范（常见是 `Bearer <token>`）；不一致会导致 401。
- `FormData` boundary：不要手动设置 `Content-Type`；让浏览器自动生成 boundary，避免上传失败。
- 参数校验：在 API 调用前做前端校验（必填、类型、范围），减少后端错误与无效请求。

八、常见问答（Q&A）

Q：如果未来某个接口要求额外的 Header 字段，如何添加？

- 在该调用处通过 `headers` 单次添加：

```
await request.post('/path', payload, { headers: { 'X-Extra': 'value' } })
```

- 若多数接口都需要：在拦截器或默认 headers 中统一设置。
- 若偶尔需要：给 API 封装函数加 `config` 透传参数，调用时再传入自定义 headers。

Q：`application/json` 与 `multipart/form-data` 的区别？

- JSON：结构化文本数据，适合纯文本；浏览器自动设置 `Content-Type: application/json`。
- FormData：混合文本+文件，适合上传场景；由浏览器自动设置 `Content-Type: multipart/form-data; boundary=...`。

Q：GET/DELETE 该怎么传参？

- 通过 `params` 传对象，Axios 自动序列化为查询串；DELETE 若后端强制要 body，可用 `{ data }`，但更常见是 `params`。

Q：响应是否可能不是 JSON？

- 本项目接口统一返回 JSON（包括空对象）。
- 其他类型如文件下载（`blob`、`arraybuffer`）或纯文本，当前项目未使用；如需支持，在请求时指定 `responseType` 并在拦截器或调用处特殊处理。

参考位置（代码引用）

- `src/utils/request.js:18-31` 请求拦截器设置 `Authorization`；`src/utils/request.js:35-45` 响应成功判断与业务失败提示；`src/utils/request.js:53-58` 401 处理。
- `src/api/user.js:5-12, 18-23, 24-32, 14` 用户相关接口的 JSON/patch 用法。
- `src/api/article.js:3-4, 6-16, 17-27, 29-31, 33-38, 42` 文章分类与文章管理的 Query、JSON、FormData 用法。